

Team 3
Document 2: RISC-V Modules [Documentation]

1. Module: RV32I46F5SP

Signal Table

Port Name	Width / Type	Source Module	Explanation
clk	1-bit Input	External System	The master clock signal that synchronizes all pipeline stages.
reset	1-bit Input	External System	Active-high reset signal used to initialize the processor state.

Port Name	Width / Type	Destination Module	Explanation
retire_instruction	32-bit Output	External Debugger	The instruction currently completing execution (writing back), used for verification.

Module Description: This module acts as the top-level structural wrapper that interconnects the 5-stage RISC-V pipeline components and handles core combinational routing. It instantiates all pipeline registers and datapath modules, manages the Register File write-back selection (choosing between memory loads, ALU results, immediates, PC+4, or CSR data), and uses combinational logic to route ALU inputs to resolve data hazards. It also contains logic to force a hardcoded debug instruction if the processor enters debug mode.

2. Module: ProgramCounter

Signal Table

Port Name	Width / Type	Source Module	Explanation
clk	1-bit Input	System Clock	The clock edge trigger for updating the PC value.
reset	1-bit Input	System Reset	Resets the PC to the boot address (0x0).
next_pc	32-bit Input	PC Controller	The calculated address of the next instruction to fetch.

Port Name	Width / Type	Destination Module	Explanation
pc	32-bit Output	Instruction Memory / PC+4	The current address of the instruction being executed.

Module Description: A synchronous 32-bit register holding the memory address of the current instruction being fetched. On the rising edge of the clock, if the reset signal is active, it clears the PC to 0. Otherwise, it updates its internal register to the address provided by the PC Controller. This updated address is then broadcast to the Instruction Memory to fetch the code and down the pipeline for tracking.

3. Module: PCPlus4

Signal Table

Port Name	Width / Type	Source Module	Explanation
pc	32-bit Input	Program Counter	The current Program Counter value.

Port Name	Width / Type	Destination Module	Explanation
pc_plus_4	32-bit Output	IF/ID Register / PC Controller	The address of the next sequential instruction (pc + 4).

Module Description: This is a dedicated, purely combinational arithmetic module that calculates the default sequential address for the next instruction. It continuously adds 4 to the current pc value (assuming standard 32-bit RISC-V instructions). This calculated value is used for normal sequential execution and for saving return addresses during jump-and-link instructions.

4. Module: PCController

Signal Table

Port Name	Width / Type	Source Module	Explanation
jump	1-bit Input	Control Unit	Signal indicating an unconditional jump (JAL/JALR).

branch_estimation	1-bit Input	Branch Predictor	Prediction signal indicating if a branch should be taken.
branch_prediction_miss	1-bit Input	Branch Logic	Signal indicating the previous prediction was incorrect.
trapped	1-bit Input	Trap Controller	Signal indicating a system trap or exception occurred.
pc_stall	1-bit Input	Hazard Unit	Signal to pause PC updates (e.g., during hazard stalls).
pc	32-bit Input	Program Counter	Current PC value, used when stalling.
jump_target	32-bit Input	ALU	Target address for jump instructions.
branch_target	32-bit Input	Branch Predictor	Predicted target address for branches.
branch_target_actual	32-bit Input	Branch Logic	Corrected target address for mispredicted branches.
trap_target	32-bit Input	Trap Controller	Vector address for handling traps/exceptions.

Port Name	Width / Type	Destination Module	Explanation
next_pc	32-bit Output	Program Counter	The final decided address for the next cycle.

Module Description: This module acts as a priority multiplexer that decides the exact address the ProgramCounter should fetch next. It evaluates various processor states using strict combinational priority: it will stall the PC if commanded, force a trap target if an exception occurs, route to a jump target, correct the flow to an actual branch target if a misprediction occurred, fetch a speculative branch target if predicted, and finally default to $pc + 4$ if no other control flow changes are active.

5. Module: PCAligner

Signal Table

Port Name	Width / Type	Source Module	Explanation
raw_next_pc	32-bit Input	ALU / Branch Logic	The potentially unaligned target address.

Port Name	Width / Type	Destination Module	Explanation
next_pc	32-bit Output	PC Controller	The 4-byte aligned address ready for the PC.

Module Description: Hardware safety logic designed to guarantee word-aligned memory access. It uses combinational logic to force the two least significant bits (LSBs) of the incoming address to 00. This ensures instructions are always fetched from 4-byte boundaries, natively preventing misaligned instruction fetch exceptions.

Note: This module is defined in the provided code but not explicitly instantiated in the RV32I46F5SP top-level routing.

6. IF_ID_Register Module

Signal Table

Port Name	Width / Type	Source Module	Explanation
clk	1-bit Input	System Clock	Clock signal for synchronising the pipeline stage.
reset	1-bit Input	System Reset	Resets the register contents to default (NOP).
flush	1-bit Input	Hazard Unit	Clears the register (inserts a bubble) when high.
IF_ID_stall	1-bit Input	Hazard Unit	Holds current value (freezes pipeline) when high.
IF_pc	32-bit Input	Program Counter	PC value from the Fetch stage.
IF_pc_plus_4	32-bit Input	PC Plus 4	Next sequential PC from the Fetch stage.
IF_instruction	32-bit Input	Instruction Memory	Raw instruction fetched from memory.
IF_branch_estimation	1-bit Input	Branch Predictor	Prediction bit from the Fetch stage.

Port Name	Width / Type	Destination Module	Explanation
ID_pc	32-bit Output	ID/EX Register	PC value passed to the Decode stage.
ID_pc_plus_4	32-bit Output	ID/EX Register	Next sequential PC passed to the Decode stage.
ID_instruction	32-bit Output	Instruction Decoder	Instruction passed to the Decode stage.
ID_branch_estimation	1-bit Output	ID/EX Register	Prediction bit passed to the Decode stage.

Module Description: The synchronous pipeline boundary separating the Instruction Fetch (IF) and Instruction Decode (ID) stages. On the clock edge, it captures fetched IF data and passes it to the Decode stage logic. If a control hazard occurs (triggering a flush or reset), it clears its PC tracking and injects a RISC-V NOP (32'h0000_0013) into the instruction stream to safely bubble the pipeline. If a stall is active, it ignores new inputs and holds its current values to pause execution.

DECODE

Instruction Decoder (InstructionDecoder.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation

instruction	32 bits	IF/ID Register	Fetch machine code instruction to be decoded
--------------------	---------	----------------	--

Output Ports:

Port Name	Width	Destination Module	Explanation
opcode	7 bits	Control Unit / ID Logic	Major operation code extracted from instruction[6:0]
funct3	3 bits	Control Unit / ALU Control	Function field 3 extracted from instruction[14:12]
funct7	7 bits	Control Unit / ALU Control	Function field 7 extracted from instruction[31:25]
rs1	5 bits	Register File	Source register 1 index extracted from instruction[19:15]
rs2	5 bits	Register File	Source register 2 index extracted from instruction[24:20]
rd	5 bits	ID/EX Register	Destination register index extracted from instruction[11:7]
raw_imm	20 bits	Immediate Generator	Extracted unextended immediate bits (scrambled based on type)

Explanation:

The Instruction Decoder is a combinational logic block responsible for parsing a raw 32-bit machine language instruction into its control and data fields. It identifies the operation type (R, I, S, B, U, J) and extracts register identifiers and immediate fragments. The module serves as the primary entry point for the Decode stage, mapping the 32-bits of the incoming instruction to specific architectural signals required by the Control Unit and Register File to execute the command correctly.

Immediate Generator (ImmediateGenerator.v)**Input Ports:**

Port Name	Width / Type	Source Module	Explanation
raw_imm	20 bits	Instruction Decoder	Raw scrambled immediate bits from the decoder
opcode	7 bits	Instruction Decoder	Opcode used to determine the sign-extension format

Output Ports:

Port Name	Width	Destination Module	Explanation
imm	32 bits	ID/EX Register	Fully formatted, sign-extended 32-bit immediate value

Explanation:

The Immediate Generator is responsible for taking the raw, scrambled bits extracted by the decoder and formatting them into a full 32-bit, sign-extended value suitable for arithmetic operations. Since RISC-V immediate values are variable in length and encoding (e.g., branch offsets are multiples of 2), this module standardizes the immediate output to 32 bits regardless of the instruction type, ensuring negative numbers are correctly preserved via sign extension.

Instruction Cache (InstructionCache.v)**Input Ports:**

Port Name	Width / Type	Source Module	Explanation
clk	1 bit	Top Module	System clock for synchronous operations
reset	1 bit	Top Module	System reset to clear valid bits and LRU
address	10 bits	Program Counter (PC)	Read address containing Index and Tag
update_enable	1 bit	Memory Controller	Enables writing to cache when a miss occurs
im_data	32 bits	Main Memory	Instruction data fetched from main memory

Output Ports:

Port Name	Width	Destination Module	Explanation
hit	1 bit	Hazard Unit / Controller	Indicates if the requested instruction was found in cache
data	32 bits	IF/ID Register	The 32-bit instruction data retrieved from cache

Explanation:

This module implements a small, high-speed memory unit designed as a 2-way set-associative cache to mitigate the speed gap between the processor and main memory. It organizes data into sets and "ways," using an address tag to verify if the data belongs to the requested address. The module includes logic for hit detection and a Least Recently Used (LRU) policy to manage data replacement, ensuring that frequently accessed instructions are kept close to the processor to reduce latency.

ID/EX Pipeline Register (ID_EX_Register.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
clk	1 bit	Top Module	System clock
reset	1 bit	Top Module	Synchronous reset
flush	1 bit	Hazard Unit	Flushes register content (inserts NOP)

ID_EX_stall	1 bit	Hazard Unit	Stalls the pipeline (holds previous value)
ID_pc	32 bits	IF/ID Register	Program Counter from ID stage
ID_pc_plus_4	32 bits	ID Adder	Next PC (PC + 4) from ID stage
ID_branch_estimation	1 bit	Branch Predictor	Branch prediction bit from ID stage
ID_instruction	32 bits	IF/ID Register	The instruction word being processed
ID_jump	1 bit	Control Unit	Jump control signal
ID_branch	1 bit	Control Unit	Branch control signal
ID_alu_src_A_select	2 bits	Control Unit	Select signal for ALU operand A
ID_alu_src_B_select	3 bits	Control Unit	Select signal for ALU operand B
ID_memory_read	1 bit	Control Unit	Memory read enable signal

ID_memory_write	1 bit	Control Unit	Memory write enable signal
ID_register_file_write_data_select	3 bits	Control Unit	Select signal for Write Back data
ID_register_write_enable	1 bit	Control Unit	Register file write enable signal
ID_csr_write_enable	1 bit	Control Unit	CSR write enable signal
ID_opcode	7 bits	Instruction Decoder	Opcode passed from ID stage
ID_funct3	3 bits	Instruction Decoder	Funct3 field passed from ID stage
ID_funct7	7 bits	Instruction Decoder	Funct7 field passed from ID stage
ID_rd	5 bits	Instruction Decoder	Destination register address
ID_raw_imm	20 bits	Instruction Decoder	Raw immediate bits

ID_read_data1	32 bits	Register File	Data read from source register 1
ID_read_data2	32 bits	Register File	Data read from source register 2
ID_rs1	5 bits	Instruction Decoder	Source register 1 address
ID_rs2	5 bits	Instruction Decoder	Source register 2 address
ID_imm	32 bits	Immediate Generator	Sign-extended immediate value
ID_csr_read_data	32 bits	CSR Unit	Data read from CSR register

Output Ports:

Port Name	Width	Destination Module	Explanation
EX_pc	32 bits	EX Stage	PC passed to Execute stage

EX_pc_plus_4	32 bits	EX Stage	Next PC passed to Execute stage
EX_branch_estimation	1 bit	EX Branch Logic	Branch estimation passed to EX
EX_instruction	32 bits	EX Stage	Instruction word passed to EX
EX_jump	1 bit	EX Stage	Jump signal passed to EX
EX_memory_read	1 bit	EX/MEM Register	Memory read signal passed to EX
EX_memory_write	1 bit	EX/MEM Register	Memory write signal passed to EX
EX_register_file_write_data_select	3 bits	EX/MEM Register	WB selection signal passed to EX
EX_register_write_enable	1 bit	EX/MEM Register	Reg write enable passed to EX
EX_csr_write_enable	1 bit	EX/MEM Register	CSR write enable passed to EX

EX_branch	1 bit	EX Branch Logic	Branch signal passed to EX
EX_alu_src_A_select	2 bits	ALU Mux	ALU A select signal passed to EX
EX_alu_src_B_select	3 bits	ALU Mux	ALU B select signal passed to EX
EX_opcode	7 bits	ALU Control	Opcode passed to EX
EX_funct3	3 bits	ALU Control	Funct3 passed to EX
EX_funct7	7 bits	ALU Control	Funct7 passed to EX
EX_rd	5 bits	Forwarding Unit / EX/MEM	Dest register addr passed to EX
EX_raw_imm	20 bits	EX Stage	Raw immediate passed to EX
EX_read_data1	32 bits	ALU / Forwarding Unit	RS1 data passed to EX
EX_read_data2	32 bits	ALU / Forwarding Unit	RS2 data passed to EX

EX_rs1	5 bits	Forwarding Unit	RS1 address passed to EX
EX_rs2	5 bits	Forwarding Unit	RS2 address passed to EX
EX_imm	32 bits	ALU	Extended immediate passed to EX
EX_csr_read_data	32 bits	EX Stage	CSR data passed to EX

Explanation:

The ID/EX Pipeline Register is a synchronous state element that separates the Instruction Decode (ID) stage from the Execute (EX) stage. It captures all control signals (generated by the Control Unit) and data signals (from the Register File and Immediate Generator) and holds them stable for the EX stage, allowing the pipeline to process multiple instructions simultaneously. The module features specific logic for flushing the register (inserting NOPs) during control hazards and stalling updates during data hazards.

ControlUnit

Input ports:

Port Name	Width / Type	Source Module	Explanation
write_done	1-bit / Input	External / Datapath	A status flag indicating if a pending write operation has completed.
trap_done	1-bit / Input	TrapController	A status flag indicating that Pre-Trap Handling has successfully finished.
csr_ready	1-bit / Input	CSRFile	A status flag indicating the Control and Status Register file is ready for operations.
IF_ID_stall	1-bit / Input	HazardUnit	A control signal indicating that the Instruction Fetch / Decode pipeline stage is currently stalled.
opcode	7-bit / Input	InstructionDecoder	The primary 7-bit operational code from the fetched instruction, used to determine the instruction type.
funct3	3-bit / Input	InstructionDecoder	A secondary 3-bit function code from the instruction, used for finer decoding (e.g., distinguishing specific I-Type or CSR instructions).

Output ports:

Port Name	Width / Type	Destination Module	Explanation
jump	1-bit / Output	PCController / Datapath	Asserts high when an unconditional jump instruction (JAL, JALR) is decoded.
branch	1-bit / Output	BranchLogic / Datapath	Asserts high when a conditional branch instruction (BEQ, BNE, etc.) is decoded.
alu_src_A_select	2-bit / Output	ALU Input A Multiplexer	Dictates the data source for ALU input A (e.g., Register rs1, Program Counter, or zero).

alu_src_B_select	3-bit / Output	ALU Input B Multiplexer	Dictates the data source for ALU input B (e.g., Register rs2, Immediate, Shift Amount, or CSR data).
csr_write_enable	1-bit / Output	CSRFile / Datapath	Enables writing new data into the Control and Status Registers.
register_file_write	1-bit / Output	RegisterFile	Enables writing data back into the destination register (rd) of the main Register File.
register_file_write_data_select	3-bit / Output	Write-Back Multiplexer	Selects the source of the data to be written to the Register File (e.g., ALU result, Memory Load, Immediate, or PC+4).

memory_read	1-bit / Output	DataMemory / ByteEnableLogic	Asserts high to instruct the Data Memory to perform a read operation (used for LOAD instructions).
memory_write	1-bit / Output	DataMemory / ByteEnableLogic	Asserts high to instruct the Data Memory to perform a write operation (used for STORE instructions).
pc_stall	1-bit / Output	PCController	Asserts high to freeze the Program Counter when hardware is unready or a pipeline stall is active.

Module Description: The ControlUnit is a purely combinational datapath module responsible for orchestrating the behaviour of the entire processor based on the instruction currently being decoded. It acts as the "brain" of the instruction execution phase. By analysing the 7-bit opcode and the 3-bit funct3 fields extracted from the instruction by the Instruction Decoder, it generates all the necessary multiplexer selection lines, memory access flags, and write-enable signals for the ALU, Register File, Data Memory, and Control and Status Registers (CSRs).

Internally, it utilises a large case statement mapped to standard RISC-V ISA opcodes (such as Load, Store, R-Type, I-Type, Branches, and Jumps). For example, if a LOAD opcode is detected, it configures the ALU to add the base register (RD1) and the immediate value (IMM), turns on memory_read, asserts register_file_write, and sets the register write-back multiplexer to accept data from memory

(RF_WD_LOAD). Additionally, the control unit manages processor stalls; it continuously evaluates readiness signals (write_done, trap_done, csr_ready) and pipeline hazard flags (IF_ID_stall) to assert pc_stall, freezing the instruction fetch stage if downstream modules are not ready to proceed.

Miscellaneous Modules

HazardUnit (HazardUnit.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
clk	1 bit	Top Module	System clock used for updating internal registers.
reset	1 bit	Reset Controller	Resets internal tracking registers.
trap_done	1 bit	Trap_Controller	Indicates trap handling is finished.
csr_ready	1 bit	CSR_File	Shows CSR operations are complete.
standby_mode	1 bit	Exception_Detector	Pauses pipeline during exception handling.
trap_status	3 bits	Trap_Controller	Provides current trap state information.
misaligned_instruction_flush	1 bit	Exception_Detector	Indicates instruction alignment error.
misaligned_memory_flush	1 bit	Exception_Detector	Indicates memory alignment error.
pth_done_flush	1 bit	Trap_Controller	Indicates pre-trap handling completion.
ID_rs1	5 bits	IF_ID_Pipeline_Reg	Source register 1 from Decode stage.
ID_rs2	5 bits	IF_ID_Pipeline_Reg	Source register 2 from Decode stage.
ID_raw_imm	12 bits	Instruction_Decoder	Immediate value from instruction.
EX_rd	5 bits	ID_EX_Pipeline_Reg	Destination register in EX stage.
EX_opcode	7 bits	ID_EX_Pipeline_Reg	Opcode of instruction in EX stage.
EX_rs1	5 bits	ID_EX_Pipeline_Reg	Source register 1 in EX stage.
EX_rs2	5 bits	ID_EX_Pipeline_Reg	Source register 2 in EX stage.

EX_imm	12 bits	Immediate_Generator	Immediate value used in EX stage.
EX_csr_write_enable	1 bit	Control_Unit	Indicates EX stage writes to CSR.
EX_jump	1 bit	Branch_Logic	Indicates jump instruction execution.
MEM_rd	5 bits	EX_MEM_Pipeline_Reg	Destination register in MEM stage.
MEM_reg_write_en	1 bit	Control_Unit	Indicates MEM stage writes to register file.
MEM_csr_write_en	1 bit	Control_Unit	Indicates MEM stage writes to CSR.
MEM_csr_addr	12 bits	EX_MEM_Pipeline_Reg	CSR address written in MEM stage.
WB_rd	5 bits	MEM_WB_Pipeline_Reg	Destination register in WB stage.
WB_reg_write_en	1 bit	Control_Unit	Indicates WB stage writes to register file.
WB_csr_write_en	1 bit	Control_Unit	Indicates WB stage writes to CSR.
WB_csr_addr	12 bits	MEM_WB_Pipeline_Reg	CSR address written in WB stage.
branch_pred_miss	1 bit	Branch_Predictor	Indicates incorrect branch prediction.

Output Ports:

Port Name	Width	Destination Module	Explanation
hazard_mem	2 bits	ForwardUnit	ALU operand forwarding from MEM stage.
hazard_wb	2 bits	ForwardUnit	ALU operand forwarding from WB stage.
csr_hazard_mem	1 bit	CSR_File	CSR dependency with MEM stage.
csr_hazard_wb	1 bit	CSR_File	CSR dependency with WB stage.
store_hazard_mem	1 bit	ForwardUnit	Store data forwarding from MEM stage.
store_hazard_wb	1 bit	ForwardUnit	Store data forwarding from WB stage.
IF_ID_flush	1 bit	IF_ID_Pipeline_Reg	Clears instruction in IF/ID stage.
ID_EX_flush	1 bit	ID_EX_Pipeline_Reg	Clears instruction in ID/EX stage.
EX_MEM_flush	1 bit	EX_MEM_Pipeline_Reg	Clears instruction in EX/MEM stage.
MEM_WB_flush	1 bit	MEM_WB_Pipeline_Reg	Clears instruction in MEM/WB stage.
IF_ID_stall	1 bit	IF_ID_Pipeline_Reg	Stops instruction fetch/decode.
ID_EX_stall	1 bit	ID_EX_Pipeline_Reg	Pauses execution at EX entry.
EX_MEM_stall	1 bit	EX_MEM_Pipeline_Reg	Pauses MEM stage pipeline flow.
MEM_WB_stall	1 bit	MEM_WB_Pipeline_Reg	Pauses WB stage pipeline flow.

Explanation:

The HazardUnit checks if instructions in different pipeline stages depend on each other. It compares source and destination registers to detect data hazards and generates forwarding signals to avoid incorrect results. This allows the processor to reuse data from later stages instead of waiting for register updates.

The module also detects CSR access conflicts and prevents incorrect CSR reads or writes. It monitors control signals related to traps and branch prediction to decide when instructions must be removed from the pipeline. This helps maintain correct execution flow.

The HazardUnit controls pipeline stalls and flushes whenever exceptions, traps, or incorrect branch predictions occur. It ensures instructions are executed in the correct order and prevents invalid instructions from progressing. This improves processor reliability and stability.

ForwardUnit (Forward_Unit.v)

Input Ports:

Port Name	Width	Source Module	Explanation
hazard_mem	2 bits	HazardUnit	Forwarding requirement from MEM stage.
hazard_wb	2 bits	HazardUnit	Forwarding requirement from WB stage.
store_hazard_mem	1 bit	HazardUnit	Store data forwarding required from MEM.
store_hazard_wb	1 bit	HazardUnit	Store data forwarding required from WB.
MEM_imm	32 bits	EX_MEM_Pipeline_Reg	Immediate value (e.g., LUI) in MEM stage.
MEM_alu_result	32 bits	EX_MEM_Pipeline_Reg	ALU result produced in MEM stage.
MEM_csr_read_data	32 bits	CSR_File	CSR read value in MEM stage.
byte_enable_logic_rf_wd	32 bits	Byte_Enable_Logic	Load data after byte selection/extension.
MEM_pc_plus_4	32 bits	EX_MEM_Pipeline_Reg	PC+4 value in MEM stage.
MEM_opcode	7 bits	EX_MEM_Pipeline_Reg	Opcode to identify instruction in MEM.
WB_imm	32 bits	MEM_WB_Pipeline_Reg	Immediate value (e.g., LUI) in WB stage.
WB_alu_result	32 bits	MEM_WB_Pipeline_Reg	ALU result in WB stage.

WB_csr_read_data	32 bits	CSR_File	CSR read value in WB stage.
WB_be_logic_rf_wd	32 bits	Byte_Enable_Logic	Load data after byte selection in WB stage.
WB_pc_plus_4	32 bits	MEM_WB_Pipeline_Reg	PC+4 value in WB stage.
WB_opcode	7 bits	MEM_WB_Pipeline_Reg	Opcode to identify instruction in WB.
csr_hazard_mem	1 bit	HazardUnit	CSR hazard caused by MEM stage.
csr_hazard_wb	1 bit	HazardUnit	CSR hazard caused by WB stage.
MEM_csr_write_data	32 bits	CSR_File	CSR data being written in MEM stage.
WB_csr_write_data	32 bits	CSR_File	CSR data being written in WB stage.
csr_read_data	32 bits	CSR_File	Normal CSR read output (no hazard).

Output Ports:

Port Name	Width	Destination Module	Explanation
alu_forward_source_data_a	32 bits	ALU_Source_MUX_A	Forwarded operand data for ALU input A.
alu_forward_source_data_b	32 bits	ALU_Source_MUX_B	Forwarded operand data for ALU input B.
alu_forward_source_select_a	2 bits	ALU_Source_MUX_A	Select signal for ALU input A source.ID_EX
alu_forward_source_select_b	2 bits	ALU_Source_MUX_B	Select signal for ALU input B source.
store_forward_data	32 bits	Data_Memory	Forwarded data for store instructions.
store_forward_enable	1 bit	Data_Memory	Enables forwarding of store data.
csr_forward_data	32 bits	CSR_File	Forwarded CSR data during hazards.

Explanation:

The ForwardUnit provides data forwarding to prevent pipeline stalls caused by data dependencies. It selects the correct data from the MEM or WB stage when the required data is not yet written back to the register file. This allows dependent instructions to execute without waiting for register updates.

The module determines which value to forward based on the instruction type using opcode decoding. It selects between ALU results, load data, CSR data, immediate values, or PC+4 values depending on the operation. It generates both forwarding data and control signals for ALU operands and store instructions.

The ForwardUnit also handles CSR forwarding when CSR write and read instructions overlap. It prioritizes forwarding from the MEM stage, then the WB stage, and finally uses normal CSR read data. This ensures the correct, continuous execution of instructions throughout the pipeline.

Trap Controller Unit (Trap_Controller.v)

Input Ports:

Port Name	Width	Source Module	Explanation
clk	1 bit	Top Module	System clock for the TrapController FSM.
reset	1 bit	Reset Controller	Global reset for FSM and internal registers.
ID_pc	32 bits	IF_ID_Pipeline_Reg	PC value of instruction in Decode stage.
EX_pc	32 bits	ID_EX_Pipeline_Reg	PC value of instruction in Execute stage.
MEM_pc	32 bits	EX_MEM_Pipeline_Reg	PC value of instruction in Memory stage.
WB_pc	32 bits	MEM_WB_Pipeline_Reg	PC value of instruction in Writeback stage.
trap_status	3 bits	Exception_Detector	Encoded trap type (misaligned, ecall, mret).
csr_read_data	32 bits	CSR_File	Data read from CSR during trap handling.

Output Ports:

Port Name	Width	Destination Module	Explanation
trap_target	32 bits	PC_Controller	Base address for trap handler or return target.
ic_clean	1 bit	Instruction_Cache	Invalidate/clean instruction cache (FENCE.I).
debug_mode	1 bit	Debug_Module	Indicates entry into debug mode (e.g., EBREAK).
csr_write_enable	1 bit	CSR_File	Requests a CSR write operation.
csr_trap_address	12 bits	CSR_File	CSR address for read/write (mtvec, mepc, etc.).
csr_trap_write_data	32 bits	CSR_File	Data to write into the selected CSR.
trap_done	1 bit	HazardUnit	Indicates Pre-Trap Handling (PTH) is finished.
misaligned_inst_flush	1 bit	HazardUnit	Flush pipeline for misaligned-instruction trap.
misaligned_mem_flush	1 bit	HazardUnit	Flush pipeline for misaligned memory access.
pth_done_flush	1 bit	HazardUnit	Wide-flush once PTH completes and target is ready.
standby_mode	1 bit	HazardUnit	Stall pipeline during multi-cycle ECALL

			handling.
--	--	--	-----------

Explanation:

The TrapController module manages exception and trap handling in the processor by coordinating with different pipeline stages and the CSR unit. It receives trap status signals and program counter values from various pipeline stages to identify where the exception occurred. Based on the trap type, it controls the writing and reading of important control and status registers such as *mepc*, *mcause*, and *mtvec*. It also generates the correct trap target address that is used to redirect program execution to the trap handler or to return from a trap.

This module uses a finite-state machine to perform Pre-Trap Handling operations in an orderly sequence. It ensures that the processor state is correctly saved before transferring control to the trap handler. The TrapController also supports special instructions such as ECALL, EBREAK, MRET, and FENCE.I trigger debug mode, cache cleaning operations, and calculate the return address when required.

Additionally, the TrapController manages pipeline control during trap execution. It sends stall and flush signals to maintain correct instruction flow and prevent incorrect instruction execution. Once trap handling is complete, it signals the pipeline to resume normal operation, ensuring reliable, consistent processor behavior during exceptional conditions.

Branch Predictor (Branch_Predictor.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
clk	1 bit	Top Module	Clock signal used to update the prediction state
reset	1 bit	Top Module	Resets branch prediction history
IF_opcode	7 bits	Instruction Fetch stage	Opcode of the instruction in IF stage
IF_pc	XLEN	Program Counter	Current program counter value
IF_imm	XLEN	Immediate Generator (IF stage)	Immediate offset for branch target calculation
EX_branch	1 bit	Execute stage	Indicates a branch instruction in EX stage
EX_branch_taken	1 bit	Branch Logic	Actual result of the branch execution

Output Ports:

Output Signal	Width	Destination Module	Description
---------------	-------	--------------------	-------------

branch_estimation	1	Instruction Fetch (IF) stage	Predictor's decision: predict branch taken or not
branch_target	XLEN	Instruction Fetch (IF) stage	Predicted next PC value

Explanation:

The branch predictor module predicts whether a branch will be taken, ensuring the pipeline runs smoothly and reducing stalls. It takes the PC, opcode, immediate, and whether a branch instruction has reached the execute stage.

A 2-bit saturating counter is used for branch prediction. The counter has four states: strongly not taken, weakly not taken, weakly taken and strongly taken. The MSB of the counter indicates whether the prediction will be taken. If the current instruction is not a branch instruction, the branch predictor disables itself and sets the branch estimation to 0. When the branch instruction reaches the execution stage, and the result is known, the states are updated based on whether the branch was taken. If the branch is taken, it moves to a more “confident” state in the taken states. If the branch was not taken, it moves to the not taken states. The counter also saturates at the extreme values, meaning it does not overflow beyond strongly taken and strongly not taken. In summary, this module tries to predict branches and adjusts its confidence based on whether the prediction was correct, reducing stalls and ensuring smooth pipeline execution.

CSR File (CSR_File.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
clk	1 bit	Top Module	Clock signal for CSR updates
reset	1 bit	Top Module	Resets all CSR registers
trapped	1 bit	Trap / Exception Logic	Indicates an exception or trap occurred
csr_write_enable	1 bit	Control Unit / Trap Controller	Enables writing to CSR registers
csr_read_address	12 bits	Decode / WB stage	Address of CSR to be read
csr_write_address	12 bits	WB / Trap Controller	Address of CSR to be written
csr_write_data	XLEN	ALU / Trap Controller	Data to write into CSR
instruction_retired	1 bit	Writeback stage	Indicates instruction completion

Output Ports:

Port Name	Width	Destination Module	Explanation
csr_read_out	XLEN	ALU / Register File	Data read from CSR register
csr_ready	1 bit	Hazard / Control Unit	Indicates CSR access completion

Explanation:

The CSR_file module manages the Control and Status Registers of the RISC_V. CSR registers are special registers used to handle exceptions, interrupts, and traps. These registers are used for monitoring and privilege control. The module receives read and write addresses to select which register to access. When a valid CSR read is requested, the pipeline is stalled, and the CSR value is placed on the CSR read out. Only a few CSRs, like **mtvec**, **mepc**, and **mcause** can be written to. The module also tracks execution using two counters. The **mcycle** register increments on every clock cycle, measuring the elapsed time of the processor. The **minstret** registers increment only when an instruction completes, thereby counting the number of instructions executed. The trapped input signal signifies if a trap or exception has occurred. It allows the CSR file to accept trap-related writes, such as exception cause or return address.

Exception Detector (Exception_Detector.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
clk	1 bit	Top Module	Clock for synchronous trap output
reset	1 bit	Top Module	Resets trap signals
ID_opcode	7 bits	ID Stage	Opcode in Instruction Decode stage
EX_opcode	7 bits	EX Stage	Opcode in Execute stage
MEM_opcode	7 bits	MEM Stage	Opcode in Memory stage
ID_funct3	3 bits	ID Stage	funct3 field from ID stage
EX_funct3	3 bits	EX Stage	funct3 field from EX stage
MEM_funct3	3 bits	MEM Stage	funct3 field from MEM stage
alu_result	2 bits	ALU (EX stage)	LSBs of calculated address
MEM_alu_result	2 bits	ALU (MEM stage)	LSBs of memory address
raw_imm	12 bits	ID Stage	Immediate for ECALL / EBREAK / CSR
EX_raw_imm	12 bits	EX Stage	Immediate field in EX stage
csr_write_enable	1 bit	Control Unit	Used for illegal CSR detection

branch_target_lsbs	2 bits	Branch Logic	LSBs of branch target address
branch_estimation	1 bit	Branch Predictor	Indicates predicted branch taken

Output Ports:

Port Name	Width	Destination Module	Explanation
trapped	1 bit	Trap Controller / Hazard Unit	Indicates an exception or trap occurred
trap_status	3 bits	Trap Controller	Encodes the reason for the trap

Explanation:

This module is responsible for detecting exceptions and traps that can occur during instruction execution. It detects if something illegal or misaligned happens, which requires trap handling. The module observes instructions at the instruction, execution, and memory stages. The module ensures that exceptions are caught at the right time and executed in priority order. Priority is higher if the exception happens later in the pipeline. During the ID stage, the module checks for special instructions such as ECALL, EBREAK, and MRET by examining the instruction's opcode, funct3, and immediate. It also detects misaligned branch targets, where the target is not properly aligned. For the EX-stage, the module mainly checks for any address alignment issues during execution. For example, if the instruction is load word or store word, the offset must be a multiple of 4. Similarly, JAL, JALR, and the MEM stage are also checked. The output signal **trapped** indicates if an exception has occurred, and **trap_status** specifies the reason for the trap.

Execution

Arithmetic Logic Unit (ALU.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
src_A	32 bits	ID_EX_Register	Source operand A
src_B	32 bits	ID_EX_Register	Source operand B
alu_op	4 bits	ALU_Controller	Control signal to select the instruction

Output Ports:

Port Name	Width / Type	Destination Module	Explanation
alu_result	32 bits	RV32I46F_5SP	Result of operation
alu_zero	1 bit	Branch Logic, RV32I46F_5SP	Zero flag if alu_result is 0

Explanation:

The ALU is the main math unit in the processor's Execute stage. It takes two 32-bit inputs and a control signal from the ALU Controller to perform calculations. Using a case statement based on symbolic opcodes from the alu_op.vh header, it produces a 32-bit result and a high zero flag if that result is exactly zero. This zero flag is used by the Branch Logic to handle instructions like BEQ.

The unit handles arithmetic like ADD and SUB for calculations and address offsets, along with logical operations such as AND, OR, XOR, ABJ (AND-NOT), and BPA (passes operand A as result) for bit manipulation and CSR updates. It also performs shifts like logical left SLL, logical right SRL, and arithmetic right SRA, and comparisons like set less than, SLT and SLTU for signed and unsigned data. These results are sent to the RV32I_5SP module for further processing, such as calculating memory addresses for LOAD and STORE or jump targets for JAL and JALR.

ALU Controller (ALU_Controller.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
opcode	7 bits	ID_EX_Register	Instruction opcode
funct3	3 bits	ID_EX_Register	To choose between variants of the same type of instruction
funct7_5	1 bit	ID_EX_Register	To distinguish between ADD/SUB and SRL/SRA in R-type instructions
imm_10	1 bit	ID_EX_Register	To distinguish between SRLI/SRAI in I-type instructions

Output Ports:

Port Name	Width / Type	Destination Module	Explanation
alu_op	5 bits	ALU	Control signal to select the instruction

Explanation

The ALU Controller serves as a local decoder for the Execute stage, translating general instruction fields into specific 4-bit operation codes. In the RV32IF46.v file, it receives corresponding ports from the ID_EX_Register.v file. The ALU Controller performs decoding by analyzing the opcode, funct3, and funct7 fields to tell the ALU which math or logic operation to run. This logic is organized as a hierarchical case structure that first checks the 7-bit opcode to determine the instruction category before narrowing down the specific task. It uses header files of different instructions for symbolic representation.

For memory and jump instructions like LOAD, STORE, AUIPC, and JAL, the controller always selects the ADD operation to calculate addresses. For BRANCH instructions, it uses funct3 to choose between subtraction for equality tests or comparisons for relative magnitude. In I-Type and R-Type instructions, the controller looks at funct3 and specific bits like the 10th bit of immediate, imm_10, or the 5th bit of 7-bit function, funct7_5, to distinguish between similar operations, such as ADD versus SUB or logical versus arithmetic shifts. Finally, for CSR operations, funct3 is used to determine if the system should write, set, or clear bits using functions like CSRRW, CSRRS, or CSRRC.

Branch Logic (Branch_Logic.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
branch	1 bit	ID_EX_Register	Indicates a branch instruction
branch_estimation	1 bit	ID_EX_Register	Prediction from Fetch stage
alu_zero	1 bit	ALU	Zero flag from ALU
funct3	3 bits	ID_EX_Register	Branch condition type
pc	32 bits	ID_EX_Register	Program Counter
imm	32 bits	ID_EX_Register	Branch offset (immediate)

Output Ports:

Port Name	Width / Type	Destination Module	Explanation
branch_taken	1 bit	RV32I46F_5SP	True if branch condition is met
branch_target_actual	32 bits	RV32I46F_5SP	Calculated target address
branch_prediction_miss	1 bit	RV32I46F_5SP	Prediction misprediction flag

Explanation:

The Branch Logic module determines if conditional branch instructions—such as BEQ, BNE, BLT, BGE, BLTU, or BGEU—should be taken. It evaluates the instruction's funct3 field against the ALU's zero flag using a combinational case statement. For example, a BEQ is taken if the zero flag is high, while a BNE is taken if it is low. Similarly, comparisons like BLT and BGE rely on the flag to determine if the "less than" condition is met.

When a branch is confirmed as taken, the module calculates the actual target address as the current PC plus the immediate offset. If not taken, the target remains the next sequential instruction (PC + 4). To maintain pipeline efficiency, the module compares this actual outcome with the earlier branch estimation from the fetch stage. If the result differs from the prediction, it triggers a branch prediction miss signal, which flushes the pipeline to clear the incorrect instructions.

Execute / Memory Access Pipeline Register (EX_MEM_Register.v)

Input Ports:

Port Name	Width / Type	Source Module	Explanation
clk	1 bit	Top Module	System clock
reset	1 bit	Top Module	System reset
flush	1 bit	Hazard Unit	Pipeline flush signal
EX_MEM_stall	1 bit	Hazard Unit	Pipeline stall signal
EX_pc	32 bits	ID_EX_Register	PC from EX stage
EX_pc_plus_4	32 bits	ID_EX_Register	Next PC from EX stage
EX_instruction	32 bits	ID_EX_Register	Instruction word
EX_memory_read	1 bit	ID_EX_Register	Memory read enable
EX_memory_write	1 bit	ID_EX_Register	Memory write enable
EX_register_file_write_data_select	3 bits	ID_EX_Register	Write back data mux select
EX_register_write_enable	1 bit	ID_EX_Register	Register file write enable

EX_csr_write_enable	1 bit	ID_EX_Register	CSR write enable
EX_opcode	7 bits	ID_EX_Register	Opcode
EX_funct3	3 bits	ID_EX_Register	Funct3
EX_rs1	5 bits	ID_EX_Register	Source register 1 address
EX_rd	5 bits	ID_EX_Register	Destination register address
EX_read_data2	32 bits	ID_EX_Register	Store data (rs2 value)
EX_imm	32 bits	ID_EX_Register	Immediate value
EX_raw_imm	20 bits	ID_EX_Register	Raw immediate
EX_csr_read_data	32 bits	ID_EX_Register	CSR read data
EX_alu_result	32 bits	EX Phase	ALU result / Memory Address

Output Ports:

Port Name	Width / Type	Destination Module	Explanation
MEM_pc	32 bits	EX_MEM_Register	PC to MEM stage
MEM_pc_plus_4	32 bits	EX_MEM_Register	Next PC to MEM stage
MEM_instruction	32 bits	EX_MEM_Register	Instruction to MEM stage
MEM_memory_read	1 bit	EX_MEM_Register	Mem read enable to MEM
MEM_memory_write	1 bit	EX_MEM_Register	Mem write enable to MEM
MEM_register_file_write_data_select	3 bits	EX_MEM_Register	WB Mux sel to MEM
MEM_register_write_enable	1 bit	EX_MEM_Register	Reg write enable to MEM
MEM_csr_write_enable	1 bit	EX_MEM_Register	CSR write enable to MEM
MEM_opcode	7 bits	EX_MEM_Register	Opcode to MEM
MEM_funct3	3 bits	EX_MEM_Register	Funct3 to MEM
MEM_rs1	5 bits	EX_MEM_Register	RS1 addr to MEM
MEM_rd	5 bits	EX_MEM_Register	Dest reg addr to MEM
MEM_read_data2	32 bits	EX_MEM_Register	Store data to MEM
MEM_imm	32 bits	EX_MEM_Register	Imm to MEM
MEM_raw_imm	20 bits	EX_MEM_Register	Raw imm to MEM
MEM_csr_read_data	32 bits	EX_MEM_Register	CSR data to MEM

Explanation:

This module is a synchronous state element that separates the Execute (EX) and Memory Access (MEM) stages. It stores the ALU result, memory access addresses, store data, and destination register addresses to ensure the Memory stage has stable data while the Execute stage moves on to the next instruction.

The register uses clk and reset for synchronization, along with flush and EX_MEM_stall signals for hazard management. The flush signal clears the register by inserting a NOP during control hazards, while the stall signal holds current values if the memory stage is busy. During normal operation, the module propagates the ALU result for memory addressing or write-back, passes the rs2 register value for STORE instructions, and carries forward control signals and the destination register address (rd) to ensure correct updates in the final pipeline stages.

Module: Byte_Enable_Logic

Explanation:

The ByteEnableLogic module is responsible for correctly formatting data loaded from memory before writing it back to the register file. In a RISC-V processor, load instructions such as LB (Load Byte, signed), LBU (Load Byte, unsigned), LH (Load Half-word, signed), LHU (Load Half-word, unsigned), and LW (Load Word) require selective extraction of specific bytes or half-words from a 32-bit memory word. Since memory typically returns a full 32-bit value, the processor must determine the correct byte or half-word using the lower bits of the address, apply sign-extension for signed loads, apply zero-extension for unsigned loads, and forward the properly formatted 32-bit value to the writeback stage. This module operates entirely in the MEM stage of the pipeline and ensures correct alignment handling, ISA compliance with RISC-V load semantics, and proper data preparation before register writeback. Without this module, sub-word load instructions would result in incorrect data being written into registers.

Input Ports

Port Name	Width / Type	Source Module	Explanation
mem_read_data	32 bits	Data Memory	Raw 32-bit memory output
alu_result	32 bits	EX Stage	Memory address used for byte offset selection
funct3	3 bits	ID Stage	Determines load type (LB, LBU, LH, LHU, LW)

Output Ports:

Port Name	Width	Destination Module	Explanation
processed_load_data	32 bits	MEM/WB Register	Properly formatted and extended load data for writeback

Module : Data_Memory

Explanation:

The DataMemory module implements the processor's data storage subsystem and is accessed during the MEM stage to perform both load and store operations. Its responsibilities include reading 32-bit data from memory for load instructions, writing 8-bit, 16-bit, or 32-bit data for store instructions such as SB, SH, and SW, supporting byte-enable masking to selectively write memory bytes, detecting misaligned memory accesses, and supporting memory-mapped I/O (MMIO) operations such as UART output. For store operations, the ALU computes the target address, the register file provides the data to be stored, and the byte-enable logic ensures that only the correct bytes are written to memory. For load operations, memory returns a full 32-bit word which is then processed accordingly. The module may also generate exception signals in case of misaligned address access or access to illegal memory regions.

Input Ports:

Port Name	Width / Type	Source Module	Explanation
address	32 bits	ALU (EX Stage)	Memory address for load/store
write_data	32 bits	Register File (via pipeline)	Data to be stored in memory
write_enable	1 bit	Control Unit	Enables store operation

byte_enable	4 bits	Control Logic	Controls partial byte writes (SB, SH, SW)
-------------	--------	---------------	---

Output Ports:

Port Name	Width	Destination Module	Explanation
read_data	32 bits	ByteEnableLogic	Raw 32-bit memory read data
exception_signal	1 bit	Exception Detector	Indicates misaligned or illegal access

MODULE : Instruction_Memory

Explanation:

The InstructionMemory module implements a read-only memory (ROM) that stores program instructions and operates during the IF (Instruction Fetch) stage. Its responsibilities include receiving the Program Counter (PC), converting the PC into a memory index, fetching the 32-bit instruction, and providing optional debug ROM access. Instruction memory is typically word-aligned, meaning the instruction address is computed as PC divided by 4. This module does not modify instructions; it simply returns the stored machine code.

Input Ports

Port Name	Width / Type	Source Module	Explanation
PC	32 bits	Program Counter	Instruction address input
rom_debug_addr	32 bits	Debug Interface	Direct ROM access for debugging

Output Ports:

Port Name	Width	Destination Module	Explanation
instruction	32 bits	IF/ID Register	Instruction to decode
rom_read_data	32 bits	Debug Interface	External ROM read access

MODULE: MEM_WB_Register

Explanation:

The MEMWBRegister is a pipeline register located between the MEM and WB stages, and in a pipelined processor it is responsible for storing intermediate results at stage boundaries. This module captures all relevant outputs from the MEM stage, including ALU results, load data, and CSR read values, and it also propagates the register write control signals while handling pipeline stall and flush conditions. It ensures data stability across clock cycles, correct synchronization between stages, and proper writeback behaviour. Without this register, proper separation between pipeline stages would not be possible.

Input Ports

Port Name	Width / Type	Source Module	Explanation
mem_pc	32 bits	EX/MEM Register	Program counter value
mem_instruction	32 bits	EX/MEM Register	Current instruction
mem_alu_result	32 bits	ALU	ALU computation result
mem_load_data	32 bits	ByteEnableLogic	Formatted load data
mem_reg_write	1 bit	Control Unit	Register write enable

mem_rd	5 bits	Decode Stage	Destination register index
clk	1 bit	Top Module	Clock signal
reset	1 bit	Top Module	Pipeline reset
stall	1 bit	Hazard Unit	Pipeline stall control
flush	1 bit	Control Logic	Pipeline flush control

Output Ports:

Port Name	Width	Destination Module	Explanation
wb_pc	32 bits	WB Stage	PC value for writeback stage
wb_instruction	32 bits	WB Stage	Instruction in WB stage
wb_alu_result	32 bits	WB MUX	ALU result forwarded
wb_load_data	32 bits	WB MUX	Load data forwarded
wb_reg_write	1 bit	Register File	Write enable signal
wb_rd	5 bits	Register File	Destination register index

MODULE: Register File

Explanation:

The RegisterFile module implements the 32 general-purpose registers defined in the RISC-V ISA. It consists of 32 registers (x0–x31), each 32 bits wide, where x0 is permanently hardwired to zero. The module provides two read ports (rs1 and rs2) and one write port (rd). During the ID

stage, rs1 and rs2 are read asynchronously, while during the WB stage, rd is written synchronously on the clock edge. If the destination register rd is zero, the write operation is ignored. Forwarding may also be implemented to avoid read-after-write hazards. This module is central to instruction execution because all ALU operations depend on its outputs.

Input Ports:

Port Name	Width / Type	Source Module	Explanation
read_reg1	5 bits	Instruction Decode	rs1 index
read_reg2	5 bits	Instruction Decode	rs2 index
write_reg	5 bits	MEM/WB Register	rd index
write_data	32 bits	WB Stage MUX	Final writeback data
write_enable	1 bit	Control Unit	Enables register write
clk	1 bit	Top Module	Clock signal

Output Ports:

Port Name	Width	Destination Module	Explanation
read_data1	32 bits	EX Stage	Operand 1
read_data2	32 bits	EX Stage	Operand 2

